FODS ASSIGNMENT 6

Name – Atharva Shivaji Dabkekar	
Roll no. – 2090139	
PRN - 1251090357	
Division – S.Y A	
Batch – 2	
DATE – 18-08-2026	TIME – 8:00 TO 10:00

Question - Practical 6: Sorting Algorithms

Implement Bubble Sort, Selection Sort, Insertion Sort, Shell Sort, and Quick Sort. Analyse their best, average, and worst-case time complexities.

ANSWER:

1. **Bubble Sort :**
   CODE:

```
#include <iostream>
using namespace std;

int main() {
  int n;
  cout << "Enter number of elements: ";
  cin >> n;

  int arr[n];
  cout << "Enter the elements: ";
  for (int i = 0; i < n; i++) {
    cin >> arr[i];
  }

  for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
      if (arr[j] > arr[j + 1]) {
        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
      }
    }
  }
```

```
  cout << "Sorted array: ";
  for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
  }
  cout << endl;

  return 0;
}
```

OUTPUT:



2. **Selection Sort:**

Code:

```
#include <iostream>
using namespace std;

int main() {
  int n;
  cout << "Enter number of elements: ";
  cin >> n;

  int arr[n];
  cout << "Enter the elements: ";
  for (int i = 0; i < n; i++) {
    cin >> arr[i];
```

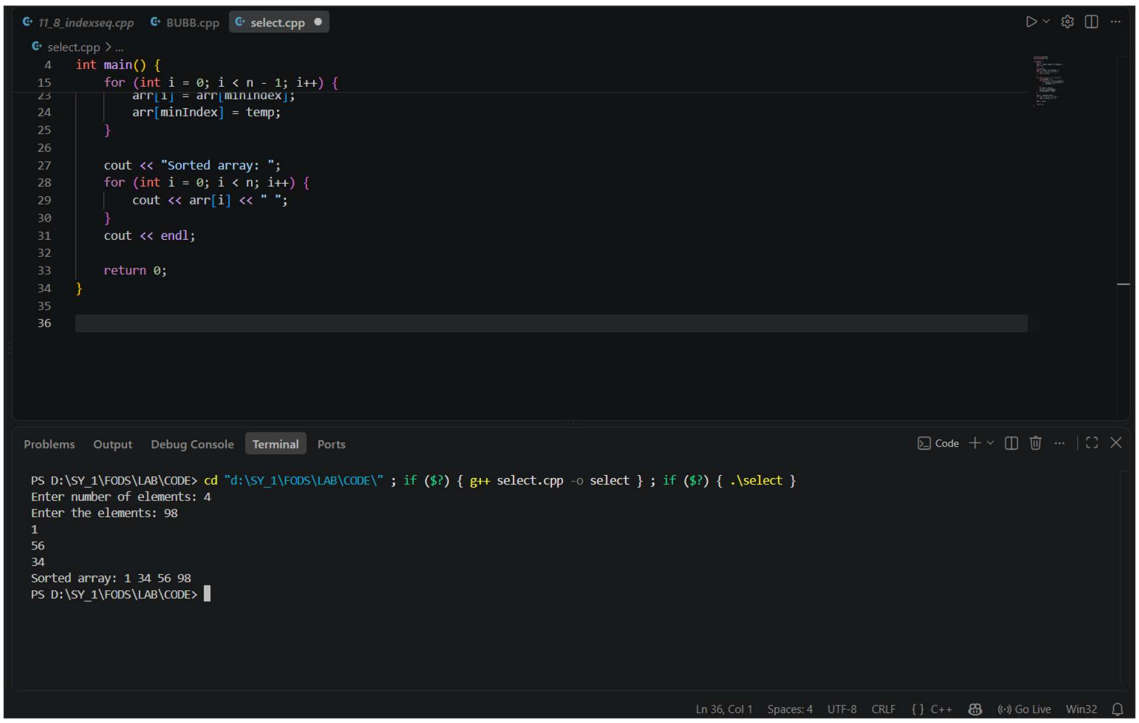
```
    }

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Output:



3. **Insertion Sort :**

CODE:

```
#include <iostream>
using namespace std;

int main() {
  int n;
  cout << "Enter number of elements: ";
  cin >> n;

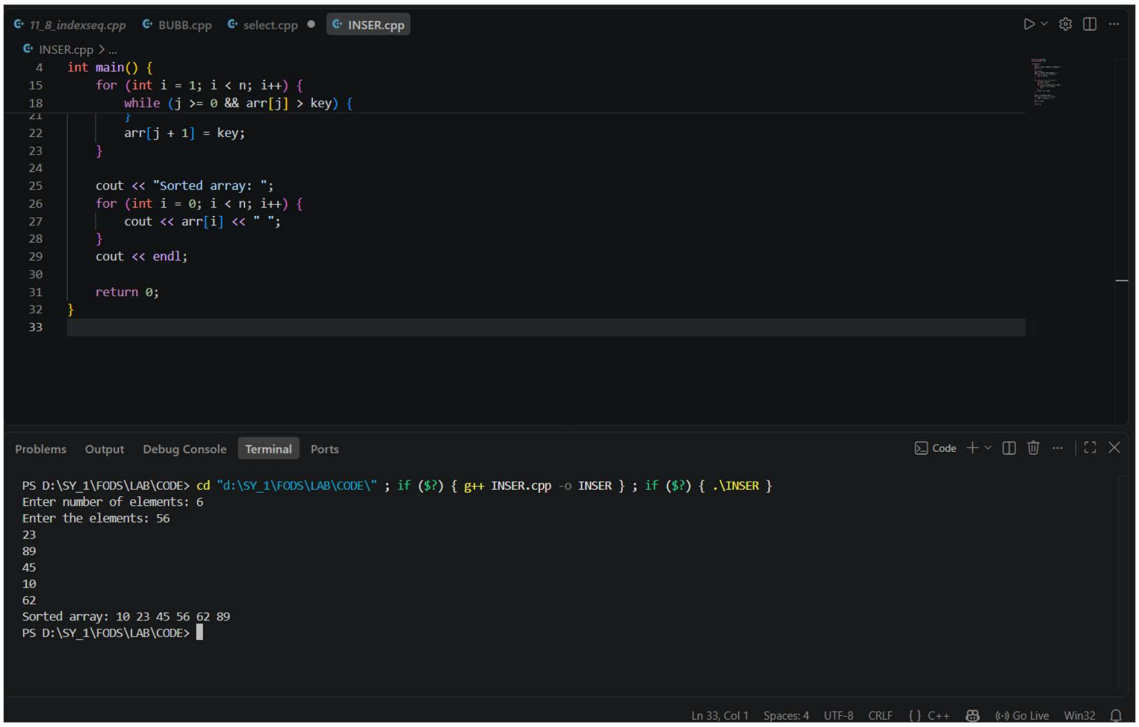
  int arr[n];
  cout << "Enter the elements: ";
  for (int i = 0; i < n; i++) {
    cin >> arr[i];
  }

  for (int i = 1; i < n; i++) {
    int key = arr[i];
    int j = i - 1;
    while (j >= 0 && arr[j] > key) {
      arr[j + 1] = arr[j];
      j--;
    }
    arr[j + 1] = key;
  }

  cout << "Sorted array: ";
  for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
  }
  cout << endl;

  return 0;
}
```

OUTPUT:



4. **Shell Sort:**

CODE:

```
#include <iostream>
using namespace std;

int main() {
  int n;
  cout << "Enter number of elements: ";
  cin >> n;

  int arr[n];
  cout << "Enter the elements: ";
  for (int i = 0; i < n; i++) {
    cin >> arr[i];
  }

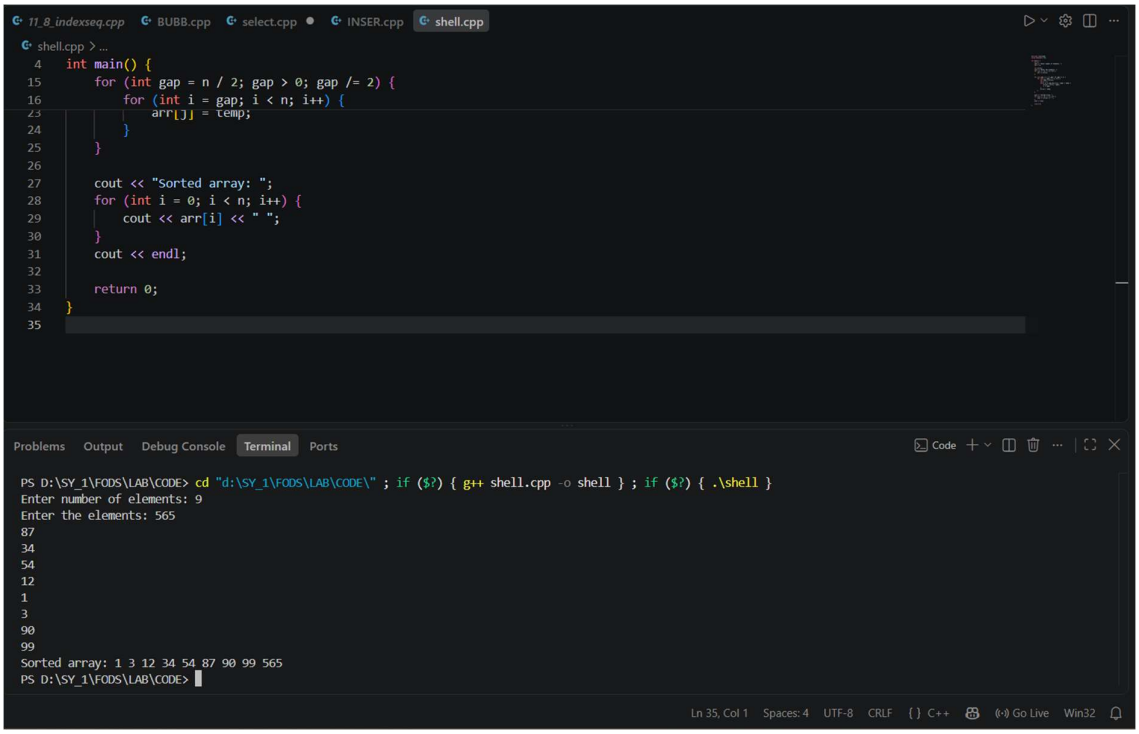
  for (int gap = n / 2; gap > 0; gap /= 2) {
    for (int i = gap; i < n; i++) {
      int temp = arr[i];
      int j = i;
      while (j >= gap && arr[j - gap] > temp) {
        arr[j] = arr[j - gap];
        j -= gap;
      }
      arr[j] = temp;
```

```
    }
  }

  cout << "Sorted array: ";
  for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
  }
  cout << endl;

  return 0;
}
```

Output:



5. **Quick Sort :**

CODE:

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
  int pivot = arr[high];
  int i = low - 1;
  for (int j = low; j < high; j++) {
    if (arr[j] < pivot) {
      i++;
      int temp = arr[i];
```

```
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    quickSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

OUTPUT:

```cpp
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
```

```
PS D:\SY_1\FODS\LAB\CODE> cd "d:\SY_1\FODS\LAB\CODE\" ; if ($?) { g++ OUICK.cpp -o OUICK } ; if ($?) { .\OUICK }
Enter number of elements: 9
Enter the elements: 8
4
5
1
67
423
89
12
55
Sorted array: 1 4 5 8 12 55 67 89 423
PS D:\SY_1\FODS\LAB\CODE>
```

Comparison of Time Complexities:

Algorithm	Best-Case	Average-Case	Worst-Case
Bubble Sort	**O(n)**	**$O(n^2)$**	**$O(n^2)$**
Selection Sort	**$O(n^2)$**	**$O(n^2)$**	**$O(n^2)$**
Insertion Sort	**O(n)**	**$O(n^2)$**	**$O(n^2)$**
Shell Sort	**O(n log n)**	**$O(n^{1.35})$**	**$O(n^2)$**
Quick Sort	**O(n log n)**	**O(n log n)**	**$O(n^2)$**